

QueueCure – Thought Process & Design Document

1. Problem Statement

Many clinics still rely on manual queue management, leading to confusion regarding token order, waiting time, and patient tracking. QueueCure was designed to digitize this process and provide real-time queue visibility.

2. Why QueueCure

The goal was to create a simple, practical, and real-world application that demonstrates queue data structures, real-time communication, state management, and persistence.

3. System Design

The system consists of a Receptionist Dashboard and a Waiting Room Dashboard. The receptionist manages the queue while patients view live updates in the waiting room.

4. Queue Data Structure

A JavaScript array is used to represent the queue. New patients are added at the end, and the next patient is served from the front, following FIFO (First In First Out) principles.

5. Token Generation Logic

Each new patient receives a unique token number using the nextToken state variable. The token increments automatically whenever a patient is added.

6. Search Logic

findIndex() is used to locate a patient's token. The position, number of patients ahead, and estimated waiting time are calculated from the index.

7. Cancellation Logic

filter() is used to remove a patient token from the queue while preserving all other queue entries. This approach avoids modifying the original array directly.

8. Wait Time Calculation

Estimated waiting time is calculated using: Patients Ahead × Average Consultation Time. This ensures the wait time is based on live queue data rather than hardcoded values.

9. Local Storage Persistence

localStorage is used to save queue data, current patient, consultation time, served count, and next token number. Data remains available even after page refresh.

10. Socket.IO Integration

Socket.IO enables real-time communication between the Receptionist Dashboard and Waiting Room. Any queue update is instantly reflected without manual refresh.

11. Edge Cases Handled

Empty patient names are prevented. Invalid token searches return 'Token Not Found'. Calling the next patient on an empty queue is blocked. Queue data persists across refreshes.

12. Future Scope

Potential enhancements include database integration, multiple doctor counters, SMS notifications, QR-code tokens, analytics dashboards, and appointment scheduling.

13. Key Learnings

This project strengthened understanding of React Hooks, state management, Socket.IO, localStorage, array methods (map, filter, findIndex), conditional rendering, and real-time application design.

Developed By

Sanskriti Parida
B.Tech CSE, KIIT University